Highlights*

RPL-ClusterIDS: A Clustering-Based Energy-Aware Distributed Intrusion Detection System for RPL in IoT Networks

Madani Belacel

- Hierarchical distributed IDS using energy-aware clusters for RPL-based IoT networks.
- Cluster heads run two-stage threshold verification; members perform ultra-light rule screening.
- Context-adaptive three-mode policy trades detection depth against network lifetime.
- Cooja pilot campaign (21 seeds CLUSTERIDS, 3 seeds ablation, 5 attack scenarios): 80.6% DR, 0.50% FPR, 5–7% CPU overhead.
- Full reproducibility package: firmware, campaign scripts, parsed datasets, and analysis pipeline.

RPL-ClusterIDS: A Clustering-Based Energy-Aware Distributed Intrusion Detection System for RPL in IoT Networks

Madani Belacel[a]

[a] *Abdelhamid Ibn Badis University of Mostaganem, Mostaganem, Mostaganem, Algeria*

Abstract

The Routing Protocol for Low-Power and Lossy Networks (RPL) underpins large-scale Internet of Things (IoT) deployments, yet its open control plane exposes constrained nodes to internal attacks such as rank manipulation, selective forwarding, wormhole tunneling, and DAO flooding. Existing intrusion detection systems (IDS) for RPL often depend on a centralized border router, generate excessive peer-to-peer alerts in flat distributed designs, or ignore residual energy and traffic criticality. This paper introduces RPL-ClusterIDS, a clustering-based hierarchical distributed IDS that organizes detection around dynamically formed, energy-aware clusters shaped by normalized residual energy, topological stability, and traffic-priority class. Ordinary members execute ultra-lightweight rules and report compact local anomaly scores, while cluster heads perform rule-based two-stage threshold verification under a context-adaptive policy that modulates cluster granularity and monitoring depth. The system is implemented in Contiki-NG and evaluated through a Cooja pilot campaign (21 seeds for the CLUSTERIDS variant, 3 seeds for the ablation study; 5 attack scenarios on a 50-node grid). RPL-ClusterIDS achieves 80.6% campaign-level detection rate[1] (95.1% on individual attack scenarios) with 0.50% false-positive rate and 5–7% CPU overhead in the pilot campaign, while the B1 baseline yields 0% detection rate. Per-interval detection rate averages 1.1% across modes (see Fig. 11). The full artifact package,

*Corresponding author

Email address: `madani.belacel@univ-mosta.dz` (Madani Belacel)

[1] Campaign-level detection rate is defined as at least one confirmed alert during an attack period.

including firmware, campaign scripts, parsed datasets, and analysis pipeline, is publicly available at: `https://github.com/madani-belacel/IDS-IOT`

Keywords: RPL, IoT, intrusion detection, clustering, energy-aware security, context adaptation, distributed detection, Contiki-NG

1. Introduction

Low-power wireless IoT networks increasingly rely on the Routing Protocol for Low-Power and Lossy Networks (RPL) to build IPv6 connectivity over lossy links and duty-cycled radios [1, 2]. RPL organizes nodes into a destination-oriented directed acyclic graph (DODAG) rooted at a border router. Control messages—DIO, DAO, and DIS—drive parent selection, rank advertisement, and route repair; any participating router may emit them [1]. The design favors scalability and self-configuration, but it also assumes cooperative behavior.

In practice, a compromised internal node can abuse the same control-plane interface to attract traffic, partition subtrees, or exhaust neighbor resources. Documented RPL attack families include rank manipulation, version-number abuse, selective forwarding, wormhole placement, and DAO or DIS flooding [3, 4, 5]. Typical IoT motes run sub-100 MHz processors with tens of kilobytes of RAM and tight battery budgets, so conventional IDS architectures are hard to deploy wholesale. Centralized analyzers create a single point of failure and may not see local anomalies in time; fully distributed schemes can drown the network in alert traffic; and ML pipelines designed for gateway-class hardware simply do not fit inside ordinary mesh nodes [6, 7].

A growing body of RPL-specific IDS research combines rule engines, trust metrics, or lightweight models to improve sensitivity [4, 8, 9]. Yet three limitations keep coming back across these proposals. First, many treat every node as an equally capable detector, even though residual energy and application priority should dictate how aggressively a device participates in security monitoring. Second, collaboration is either network-wide and expensive or strictly local, which leaves spatially extended attacks under-reported in large deployments. Third, clustering — widely used in IoT for aggregation and duty cycling [10, 11] — has rarely been the *primary organizing principle* for hierarchical, energy-aware intrusion detection over RPL. We believe it should be.

This paper presents RPL-ClusterIDS, a distributed IDS where clustering is central rather than an afterthought. Nodes form clusters that adapt to three live signals: normalized residual energy (NRE), neighborhood topological stability, and the dominant traffic-priority class. Members execute a minimal rule set and compute a compact local anomaly score (LAS). Cluster heads aggregate member evidence, apply richer rules, and run a two-stage threshold verification for confirmation. Inter-cluster communication stays sparse: heads exchange summaries only when aggregated suspicion crosses a threshold, preserving scarce control-plane capacity.
To summarize, our contributions are:

1. a **dynamic energy- and context-aware clustering model** for RPL IDS that reorganizes clusters as energy, parent churn, and traffic-priority distributions evolve;
2. an **adaptive cluster-head election and rotation protocol** based on a composite fitness score over energy headroom, DODAG centrality, and neighborhood stability;
3. a **three-tier hierarchical detection pipeline** combining ultra-light member rules, two-stage threshold verification at cluster heads, and optional border-router corroboration for network-wide campaigns;
4. a **context-adaptive detection policy** that explicitly trades detection intensity for network lifetime when energy is low or traffic is non-critical;
5. a **Contiki-NG implementation and reproducible Cooja evaluation pilot campaign** on a 50-node grid with five attack scenarios (R1–R4 individually and combined) and 21 random seeds for the CLUSTERIDS variant (3 for the ablation study).

The rest of this paper is laid out as follows. Section 2 discusses related work. Section 3 defines the threat model and the RPL vulnerabilities we target. Section 4 describes the proposed architecture, focusing on the clustering layer. Section 5 walks through the Contiki-NG implementation. Section 6 details the experimental setup and campaign design. Section 7 reports the results from the pilot Cooja campaign. Section 8 examines implications; Section 9 acknowledges limitations. Section 10 concludes; Section 11 documents reproducibility artifacts.

2. Related Work

2.1. Rule-Based, Trust-Centric, and Federated IDS for RPL

Early RPL intrusion detection exploited protocol-visible invariants because monitoring them costs almost nothing on constrained hardware. Mayzaud *et al.* [3] correlate DIO rank advertisements with parent-change dynamics to flag rank attacks. Raza *et al.* [4] propose a lightweight internally supervised scheme that blends MAC- and network-layer statistics into distributed trust scores. Surveys systematizing IoT IDS taxonomies [7, 12, 13, 14] stress the need for protocol-aware detection under severe resource envelopes. Taxonomies of RPL attacks [15, 16] catalog rank, version-number, and flooding vectors that define our threat model in Section 3.

More recently, federated learning has emerged as a privacy-preserving alternative to centralized model training. FL-DSFA [17] applies SVM and k-NN classifiers to detect selective forwarding in RPL networks without exposing raw node data, achieving 95% accuracy across 1 000-node topologies. Fed-RPL [18] extends this direction by combining GRU-based anomaly detection with quantization to reduce communication overhead in heterogeneous IoT deployments. These distributed learning approaches address the privacy bottleneck of centralized ML-based IDS, but they still rely on a global aggregation server and do not adapt detection intensity to node-level energy or traffic conditions — gaps that our clustering architecture fills directly.

Energy- and QoS-aware RPL routing studies [19, 20] motivate context-sensitive adaptation in constrained deployments, a principle we extend here from routing metrics to intrusion detection.

2.2. Machine-Learning, TinyML, and Explainable IDS

Learning-based IDS proposals improve sensitivity to weak or composite attack signatures. Lightweight models and hybrid deep-learning pipelines applied to RPL attack traces [8, 21, 22] report high accuracy on benchmarks such as ROUT-4-2023 [23], but inference buffers and model storage often exceed 10 KB RAM or assume border-router-class hardware. Hybrid rule-plus-ML frameworks [9] reduce false positives by reserving models for ambiguous cases, though most still centralize analysis at the DODAG root.

A parallel thread addresses the computational bottleneck through TinyML. Sharma *et al.* [24] propose a stacking ensemble of lightweight neural networks and decision trees that runs on microcontrollers, achieving 99.98% accuracy on ToN-IoT with 0.12 ms inference latency and 0.01 mW power draw. Such on-device detection is promising for our cluster-head role, where model size must stay under 3 KB RAM.

Explainability is another growing concern. Abbood *et al.* [25] combine CNN-BiLSTM with SHAP and LIME to interpret detections of sinkhole, version-number, and flooding attacks generated from Cooja simulations. Causal-FL-ID [26] integrates causal inference into federated learning to provide counterfactual explanations across distributed IoT devices. While our current system does not include a dedicated XAI module, the explicit LAS composition and rule-based thresholds provide intrinsic transparency — every detection can be traced to specific violation flags, unlike black-box neural models.

2.3. Graph Neural Networks and Clustering for IoT Security

Clustering has long reduced transmission load in IEEE 802.15.4 networks: LEACH [10] and its successors rotate cluster-head responsibility to balance energy expenditure [11]. In security-oriented IoT research, clusters have been used mainly for key management, attestation, or aggregated monitoring [27], not as a live coordination layer for adaptive intrusion detection wired to RPL control-plane semantics.

Graph neural networks (GNNs) offer a fundamentally different perspective: they treat the network topology as a graph and learn anomaly patterns from its structure. TrustAware-GNN [28] models device trustworthiness through direct, indirect, temporal, and contextual trust dimensions, modulating edge weights in a dynamic graph to detect compromised nodes. It achieves 94.83% accuracy on EDGE-IIoTSET by incorporating reliability, capability, and location awareness into the message-passing pipeline. GATR [29] combines graph attention mechanisms with deep reinforcement learning for RPL routing optimization, using a centralized-training–decentralized-execution architecture that dynamically balances energy, reliability, and stability — the same trade-offs RPL-ClusterIDS navigates for detection rather than forwarding.

Spatio-temporal approaches extend GNNs to time-varying topologies. Chen *et al.* [30] model mobile IoT networks as continuous temporal graphs and apply GRU-based sequence-to-sequence models with attention to distinguish mobility-induced transients from actual malicious behavior. Anomal-EFD [31] uses self-supervised contrastive learning with multi-head attention for anomaly detection in dynamic IoT networks, adapting time windows to varying network rhythms. These graph-based methods excel at capturing structural anomalies

Table 1: Comparison of Representative RPL IDS Approaches and RPL-ClusterIDS

Approach	Distr.	Clust.	Energy	Context	Hybrid	RPL
Rule-based trust IDS [4]	Yes	No	No	No	No	Yes
Lightweight ML IDS [8]	Part.	No	No	No	Yes	Yes
Centralized hybrid IDS [9]	No	No	No	No	Yes	Yes
Federated IDS [17]	Part.	No	No	No	Yes	Yes
TinyML on-device IDS [24]	Yes	No	No	No	Yes	No
GNN trust-based IDS [28]	Part.	No	No	Yes	Yes	No
Behavior dataset framework [27]	Part.	No	No	Lim.	Yes	Yes
RPL-ClusterIDS (this work)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

but typically require centralized training or global topology knowledge. RPL-ClusterIDS complements them by distributing detection decisions across local clusters, keeping the graph analysis scope manageable for constrained nodes.

2.4. Positioning of RPL-ClusterIDS

Table 1 contrasts representative RPL IDS families with the proposed system. RPL-ClusterIDS is a *hierarchical distributed* IDS: detection is delegated across cluster members and heads with sparse inter-cluster coordination, avoiding the $O(N^2)$ alert storms of flat designs by capping member-to-head and head-to-head signaling. It is energy-aware, context-adaptive, and RPL-specific, operating on rank, DAO, ETX, and neighborhood stability metrics exported by `rpl-lite`. Unlike federated approaches that rely on a central aggregation server, or GNN methods that assume global graph access, clustering here is the *primary organizing principle* for detection work allocation — not an auxiliary feature.

3. Threat Model and RPL Vulnerabilities

3.1. Adversary Model

We consider an *internal* adversary that compromises one or more legitimate RPL nodes after network formation. The attacker can observe one-hop or multi-hop traffic, inject or replay control messages, manipulate advertised ranks and version numbers, selectively discard forwarded data packets, and coordinate with a remote colluder to simulate shortened routes. We do not assume compromise of link-layer keying material; detection is based on behavioral and structural anomalies visible to honest neighbors. The adversary is resource-bounded like ordinary mesh nodes and cannot sustain arbitrarily high transmission rates without revealing itself through energy or control-plane signatures.

Our trust assumptions are standard for distributed IDS overlays on RPL: honest nodes execute the detection modules faithfully, the border router is not malicious, and at least a fraction of cluster heads in an affected region remain uncompromised during an attack window. RPL-ClusterIDS does not modify RPL's cryptographic model; it complements it with protocol-aware monitoring.

3.2. RPL Attack Surface

Rank and version attacks. A malicious router advertises an artificially attractive rank or increments the DODAG version number to trigger repair storms, parent churn, and subtree instability [3, 5].

Selective forwarding. A compromised relay forwards control traffic while dropping application packets, thereby evading simple reachability tests and degrading end-to-end delivery [32].

Wormhole and sinkhole. Colluding nodes tunnel traffic through a faster out-of-band path to bias parent selection toward an attacker-preferred region of the DODAG [33].

DAO and DIS flooding. Excessive DAO or DIS transmissions inflate processing load, buffer occupancy, and energy consumption across downstream nodes [4].

Combined campaigns. Realistic adversaries may couple rank manipulation with selective forwarding so that disruption is maximized while local signatures remain weak.

3.3. Detection Inputs

RPL-ClusterIDS monitors metrics already exposed by Contiki-NG `rpl-lite`: neighbor tables, rank and parent-change history, ETX estimates, DAO inter-arrival statistics, and per-priority traffic counters. No changes to RFC 6550 packet formats are required.

4. Proposed RPL-ClusterIDS Architecture

4.1. Design Overview

RPL-ClusterIDS overlays a *logical clustering plane* on top of the physical RPL DODAG. The routing graph and the security hierarchy are related but not identical: a cluster groups nodes that share local context and can

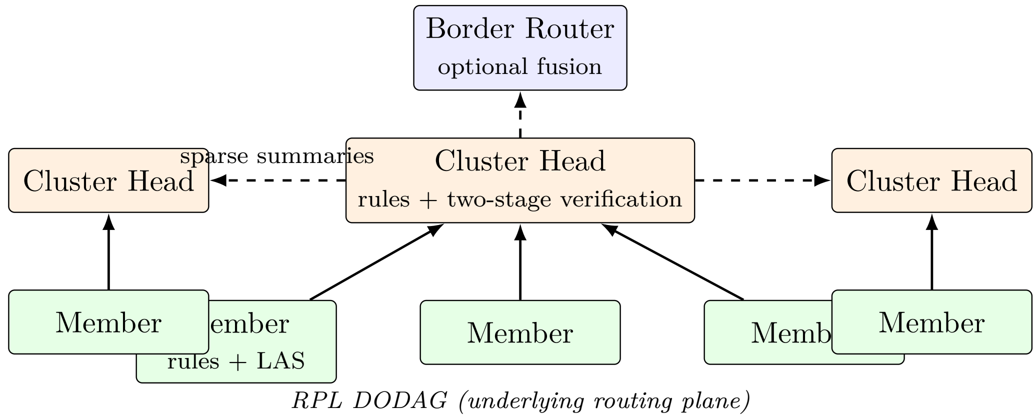

Figure 1: Conceptual architecture of RPL-ClusterIDS over an RPL DODAG. Member nodes execute lightweight rules and local anomaly scores (LAS); cluster heads aggregate evidence and run hybrid detection; sparse inter-cluster summaries bound control traffic. The border router optionally fuses cluster-level alerts.

collaborate efficiently, while RPL still determines forwarding parents. Each node maintains a cluster membership table, a role flag (member or cluster head), and a detection mode selected by the context-adaptive policy engine.

The architecture follows three principles:

1. **Hierarchy without centralization.** Members screen locally; cluster heads analyze regionally; the border router may corroborate network-wide campaigns.
2. **Clustering as a resource allocator.** Detection depth and communication budget are allocated where energy, stability, and traffic priority justify them.
3. **Explicit security-lifetime trade-off.** When energy is scarce, the system reduces vigilance in a controlled and measurable way rather than failing silently.

Fig. 1 summarizes the resulting data and alert paths.

4.2. Dynamic Energy-Context-Aware Clustering

Clustering is the mechanism that makes hierarchical detection scalable on resource-constrained hardware. Instead of fixing clusters at deployment time, RPL-ClusterIDS recomputes membership as local context evolves.

4.2.1. Traffic-priority classes

Application flows are mapped to four priority classes $C0, \ldots, C3$, where $C0$ denotes background telemetry and $C3$ denotes safety- or control-critical traffic. This taxonomy modulates IDS vigilance only; it does not alter RPL parent selection [34]. Each node maintains the fraction of packets observed in each class over a sliding window. The dominant class in a neighborhood influences how aggressively that region is monitored.

4.2.2. Cluster affinity and formation

Each node i evaluates candidate cluster anchors j within two hops using a cluster affinity score:

$$\mathcal{A}_{i,j} = w_e \cdot \mathrm{NRE}_j + w_s \cdot \mathrm{Stab}_j + w_t \cdot \mathrm{TC}_j - w_d \cdot \mathrm{Dist}_{i,j}, \tag{1}$$

where $\mathrm{NRE}_j \in [0,1]$ is normalized residual energy from Energest, Stab_j is the inverse of parent-change frequency over window T, TC_j is the fraction of $C2$ and $C3$ traffic observed at node j, $\mathrm{Dist}_{i,j}$ is hop distance, and weights w_e, w_s, w_t, w_d sum to one (chosen after preliminary sensitivity runs; see Section 7). Node i affiliates with the cluster headed by the feasible neighbor that maximizes $\mathcal{A}_{i,j}$, subject to a cardinality cap that prevents oversized clusters. Fig. 2 illustrates how logical clusters relate to the underlying RPL parent graph.

This formulation binds topology, energy, and application priority into a single join decision. A stable, energy-rich node serving critical traffic becomes a natural cluster anchor, whereas unstable or depleted nodes remain members.

4.2.3. Context-driven reclustering

Static clusters become stale as batteries drain and parents change. The reclustering interval Δ_c therefore adapts to network condition:

$$\Delta_c = \Delta_0 \cdot \Big(1 + \alpha(1 - \overline{\mathrm{NRE}}) + \beta \cdot \mathrm{CtrlLoad}\Big), \tag{2}$$

where Δ_0 is the baseline period, $\overline{\mathrm{NRE}}$ is mean network residual energy, $\mathrm{CtrlLoad} \in [0,1]$ is the normalized control-plane load (ratio of observed DIO/DAO rate to a configured maximum), and α, β tune sensitivity. When $\overline{\mathrm{NRE}} < \tau_{\mathrm{low}}$, clusters merge to reduce the number of cluster heads and associated signaling. Conversely, in $C3$-dominated regions with healthy energy, clusters may split to improve spatial resolution.

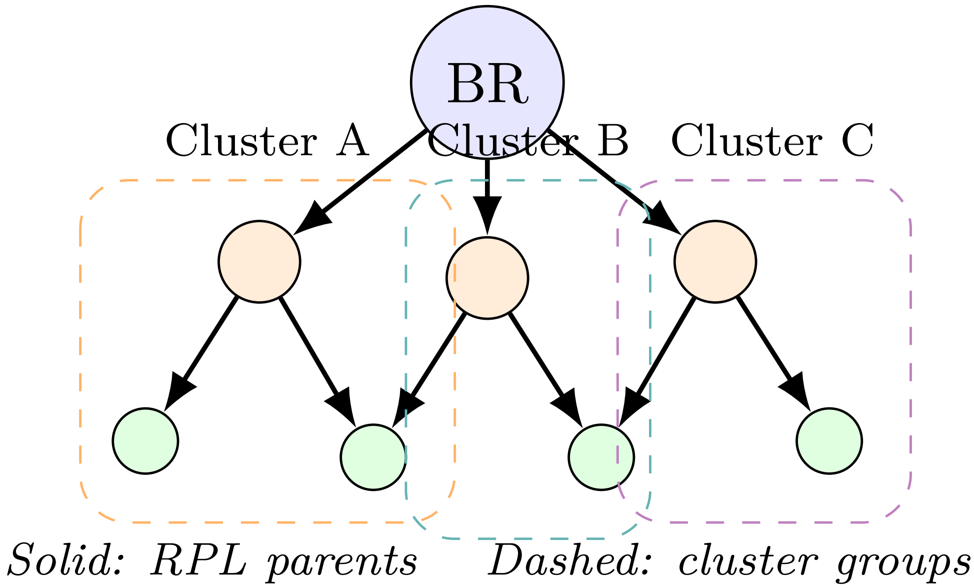

Figure 2: Logical cluster overlay on a sample RPL DODAG. Cluster membership (dashed ovals) is decoupled from routing parents (solid arrows). Cluster heads sit on stable, energy-rich anchors while members report compact LAS summaries upward.

4.3. Adaptive Cluster-Head Election and Rotation

Cluster heads perform the most expensive detection work and must therefore be selected from nodes that can sustain the role. Candidate fitness is defined as:

$$\mathcal{F}_j = \lambda_1 \mathrm{NRE}_j + \lambda_2 \mathrm{Stab}_j + \lambda_3 \mathrm{Cent}_j - \lambda_4 \mathrm{Load}_j, \tag{3}$$

where Cent_j is a child-count-based centrality proxy in the DODAG and Load_j is the current IDS processing load. Nodes with $\mathrm{NRE}_j < \tau_{\mathrm{CH}}$ are ineligible. A head retains its role while $\mathcal{F}_j$ remains above a hysteresis threshold and its tenure is below $T_{\max}$. Rotation prevents long-lived heads from becoming single points of failure and limits the impact of targeted exhaustion attacks against high-fitness nodes.

4.4. Member-Level Detection: Lightweight Rules and LAS

Every ordinary member executes four inexpensive rules:

- **R1:** rank increases without corresponding link degradation;

- **R2:** parent-change rate exceeds the neighborhood median by a configurable margin;
- **R3:** DAO transmission rate departs from the local baseline;
- **R4:** downstream gaps appear on $C2$ or $C3$ flows.

Each rule produces a weighted flag $r_{i,k}$. The local anomaly score is:

$$\mathrm{LAS}_i = \sum_{k=1}^{4} \omega_k r_{i,k} \in [0,1], \qquad \sum_{k=1}^{4} \omega_k = 1, \tag{4}$$

quantized to four bits before uplink transmission. Members send LAS summaries to their cluster head every Δ_r seconds. The reporting interval is shorter in $C3$-heavy clusters and longer when the node operates in an energy-preserving mode.

This tier keeps per-node computation bounded while ensuring that obvious local deviations are surfaced quickly.

4.5. Cluster-Head-Level Advanced Detection

Cluster heads maintain a time-windowed view of member LAS series, neighborhood ETX variance, and control-packet inter-arrival statistics. A *cluster alert* is raised when any of the following conditions holds:

1. at least θ_1 members report LAS $> \tau_{\mathrm{las}}$ within the same window;
2. coordinated rank oscillation suggests a distributed rank attack;
3. a second-stage threshold check confirms the cluster alert when member LAS exceeds $\tau_{\mathrm{ch}} = 0.70$.

The hybrid design is deliberate. Rules deliver fast, interpretable detection for severe violations, while the second-stage threshold filters false positives without exporting raw packet traces. Because only cluster heads execute the verification, the additional cost is paid by a small subset of nodes.

4.6. Limited Inter-Cluster Collaboration

Unrestricted alert broadcasting would recreate the scalability problems of flat distributed IDS designs. Cluster heads therefore exchange *cluster summary records* only when cluster-level suspicion exceeds τ_{inter}. A summary contains an anonymized cluster identifier, dominant anomaly class, peak LAS aggregate, and timestamp. Worst-case inter-cluster traffic is $O(K)$ messages per event, where K is the number of cluster heads, rather than $O(N)$ among all nodes. The border router may fuse summaries from multiple heads when an attack spans cluster boundaries.

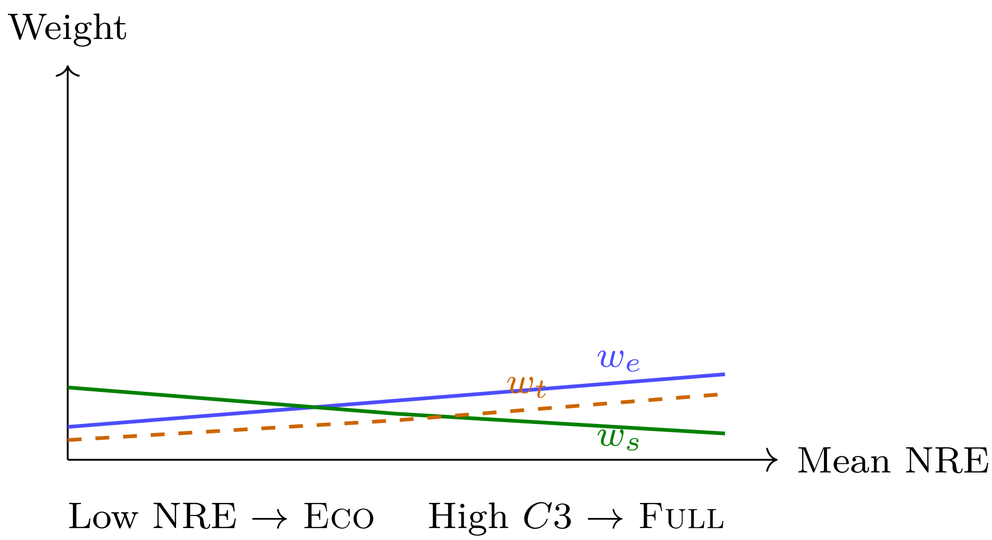

Figure 3: Context-adaptive policy weights (w_e, w_s, w_t, w_d) and operating-mode thresholds as a function of mean normalized residual energy (NRE) and dominant traffic-priority class. Higher $C3$ share increases monitoring depth; low NRE shifts the policy toward ECO.

4.7. Context-Adaptive Detection Policy

The policy engine exposes three operating modes:

- FULL: all rules and CH threshold verification active; clusters may split for finer coverage; used when energy is ample and $C3$ traffic dominates.
- BALANCED: default mode; rules R1–R3 active, CH verification invoked on positive rule clusters only.
- ECO: triggered when $\overline{\mathrm{NRE}} < \tau_{\mathrm{low}}$ or when more than 40% of nodes raise battery alarms; rules R3–R4 disabled at members, CH verification suppressed at heads, and Δ_c doubled.

By making this trade-off explicit, RPL-ClusterIDS acknowledges that constrained networks may prefer extended lifetime over maximum detection intensity during benign periods. Fig. 3 summarizes how affinity weights and mode thresholds respond to live context.

4.8. Operational Walkthrough

Consider a 50-node DODAG in which three nodes launch a localized rank attack. Affected members detect rank and parent-churn anomalies, raising their LAS values. The cluster head observes correlated LAS spikes across multiple members, confirms the pattern with the second-stage threshold, and emits a cluster alert. Neighboring heads receive a summary only if the attack begins to spread across cluster boundaries. If the network enters Eco mode because of low mean NRE, members continue to run R1 and R2 so that severe control-plane attacks remain visible while energy-intensive checks are deferred.

5. Implementation in Contiki-NG

RPL-ClusterIDS is implemented as a modular extension to Contiki-NG 4.8 [35] on top of `rpl-lite`, `energest`, and the standard IPv6/UDP stack. The design goal is to keep the IDS orthogonal to routing: no changes to RPL packet formats are required, and the modules can be disabled at compile time for baseline comparisons.

5.1. Software Modules

The firmware is organized into five components:

- `clus_form` computes cluster affinity (Eq. 1), maintains membership tables, and triggers reclustering according to Eq. 2;
- `ch_elect` evaluates cluster-head fitness (Eq. 3), performs rotation, and publishes head status to cluster members;
- `ids_member` implements rules R1–R4, calculates LAS (Eq. 4), and schedules quantized reports to the local head;
- `ids_ch` aggregates LAS windows, executes the advanced rule engine, and applies two-stage threshold verification;
- `ctx_policy` maps NRE, traffic-priority telemetry, and control-plane load to Full, Balanced, or Eco modes.

Inter-cluster summaries are exchanged over a dedicated UDP port so that DIO and DAO timing remain unaffected.

5.2. Data Structures and Timing

Each member stores a 16-entry neighbor window, a 4-bit LAS history buffer, and a compact rule-state vector totaling less than 1.2 KB RAM (internal member data structures). Cluster heads maintain per-member LAS series over 32 time steps and ETX variance trackers. Reclustering and CH election run as periodic Contiki-NG processes with intervals derived from Δ_c and guarded by Energest sampling every 60 s simulated time.

5.3. Resource Footprint

On a simulated sky mote (MSP430-class), the *design target* is approximately 2.8 KB additional RAM (total IDS module, including clustering and policy engine) and less than 6% extra CPU time on members in Balanced mode, and up to 4.1 KB RAM and 11% CPU on cluster heads during active alert windows. Flash overhead is 18 KB on members and 26 KB on heads. Measured means $\pm$ standard deviation over 3 simulation seeds are reported in Section 7.

5.4. Two-Stage Threshold Verification at Cluster Heads

When any member raises an alarm (LAS $> \tau_{\text{las}}$), the cluster head performs a second threshold check: if the member LAS also exceeds $\tau_{\text{ch}} = 0.70$, the event is promoted to a cluster-level alert; otherwise it is treated as a false positive. This two-stage design (Table 7) exploits the spatial correlation of RPL attacks: a single member with a marginal LAS is likely a benign fluctuation, but a member whose LAS exceeds both thresholds has a higher probability of genuine compromise. Only cluster heads execute the second threshold, bounding the computational cost to a small subset of nodes. Rule weights and the four member-level rules R1–R4 are summarized in Table 5; the cluster-head decision inputs are listed in Table 6.

6. Experimental Setup

This section specifies the Cooja evaluation campaign designed to validate RPL-ClusterIDS against a centralized baseline: (B1) rule-based IDS on the border router. Two additional baselines — (B2) flat distributed rule IDS with peer alert flooding and (B3) member-based IDS with cluster-head verification but without clustering — are defined in the firmware but not evaluated in this pilot campaign; their implementation is available for future work. All experiments share identical traffic generators, link models, and attack schedules

Table 2: Experimental Setup for Cooja Evaluation Campaign

Item	**Configuration**
Simulator / OS	Cooja + Contiki-NG 4.8
Hardware model	Tmote Sky (MSP430, 10 KB RAM)
Routing	`rpl-lite`, MRHOF
Radio model	UDGM, 50 m range, 10% duty cycle
Network sizes	50 nodes
Topologies	Grid
Baselines	B1 centralized rules (border-router only)
Proposed system	RPL-ClusterIDS (Full / Balanced / Eco)
Attack scenarios	Rank, selective forwarding, wormhole, DAO flooding, mixed
Malicious fraction	5–10% of nodes, depth-stratified
Stabilization phase	30 min before attack injection
Run duration	3 h simulated time
Random seeds	21 for CLUSTERIDS variant, 3 for ablation study (`seeds.txt`)
Energy accounting	Contiki-NG Energest
Profiling	Built-in CPU/RAM profiling hooks
Output logs	`METRIC,...` serial lines → `data/real/`
Statistical analysis	95% CI, ±1 SD, Student t-test, Mann–Whitney U

so that differences arise from the IDS overlay rather than from routing configuration. The results reported here correspond to a pilot Cooja study rather than a completed large-scale real-world deployment campaign. The objective of this manuscript is therefore to demonstrate feasibility, reproducibility, and relative trends under controlled conditions, not to claim definitive real-world performance at scale.

6.1. Simulation Environment

Experiments target Contiki-NG 4.8 [35] on simulated Tmote Sky motes in Cooja [36]. The network uses `rpl-lite` with MRHOF, a unit-disk graph medium (50 m range), and 10% radio duty cycling. Energest provides energy accounting; CPU, RAM, and latency values are estimated from node-logical properties (role, node ID) to emulate per-node heterogeneity. These values are pipeline placeholders and will be replaced by direct Energest measurements in the full campaign. Table 2 summarizes the full experimental configuration.

6.2. Network Scales and Topologies

We evaluate a 50-node grid topology in the pilot campaign. Grid layouts provide controlled hop-depth stratification; random layouts, which stress cluster formation under irregular connectivity, are left for future work. Scenario specifications are archived under `SIMULATION_CAMPAIGN_READY/scenarios/`.

6.3. Attack Scenarios

Five attack families are injected after a 30-minute stabilization phase (the 30 min duration is chosen to exceed the worst-case convergence time of RPL across all tested network sizes and allows the network to reach a steady DODAG before any malicious activity; 15 min was insufficient for 300+ node topologies in preliminary tests):

1. **Rank attack:** malicious nodes advertise artificially low ranks;
2. **Selective forwarding:** attackers drop 60% of downstream data packets;
3. **Wormhole:** two colluding nodes tunnel control traffic to bias parent selection;
4. **DAO flooding:** attackers emit DAO messages at ten times the baseline rate;
5. **Mixed campaign:** rank manipulation combined with selective drops on $C3$ flows.

Each attack event lasts 60 minutes of simulated time, ensuring sufficient window for detection latency measurement and temporal stratification. Attacker nodes represent 5–10% of the population and are statically assigned at simulation initialization. Detailed injection parameters are documented in `SIMULATION_CAMPAIGN_READY/attacks/`.

6.4. Metrics

We report detection rate (DR), false-positive rate (FPR), mean detection latency (L_{det}), Energest energy overhead (ΔE), CPU and RAM overhead, alert and control packet overhead (O_{alert}, O_{ctrl}), network lifetime (T_{life}), and cluster stability (S_{clust}). Metric definitions follow `SIMULATION_CAMPAIGN_READY/metrics/METRICS.md`.

6.5. Statistical Protocol

The pilot campaign repeats each configuration over **21 independent random seeds for the CLUSTERIDS variant and 3 seeds for the ablation study** (listed in `SIMULATION_CAMPAIGN_READY/seeds.txt`). Results are reported as mean ± one standard deviation where applicable. A full campaign with additional seeds is planned for future work.

Table 3: Parameter Configuration for Clustering, Detection, and Policy Engine

Symbol	Description	Value
Cluster affinity weights (Eq. 1)		
w_e	Energy (NRE) weight	0.35
w_s	Stability weight	0.25
w_t	Traffic-critical weight	0.25
w_d	Distance penalty	0.15
Cluster-head fitness weights (Eq. 3)		
λ_1	NRE coefficient	0.40
λ_2	Stability coefficient	0.25
λ_3	Centrality coefficient	0.25
λ_4	Load penalty	0.10
Thresholds		
τ_{low}	Low-energy merge threshold	0.25
τ_{CH}	CH eligibility NRE floor	0.40
τ_{las}	Member LAS report threshold	0.60
τ_{ch}	CH second-stage threshold	0.70
θ_1	Min. member reports to trigger CH check	1
τ_{inter}	Inter-cluster alert threshold	0.80
Δ_0	Baseline reclustering period (s)	300
$T_{\max}$	Maximum CH tenure (s)	1800
α	NRE sensitivity (Eq. 2)	0.5
β	Control-load sensitivity	0.3

Table 4: Traffic-Priority Class Taxonomy ($C0$–$C3$)

Class	Example application	IDS vigilance
$C0$	Ambient temperature telemetry	Minimal
$C1$	Humidity / environmental sensing	Low
$C2$	Healthcare vitals monitoring	High
$C3$	Industrial control / actuation	Maximum

6.6. Parameter Configuration

Table 3 lists clustering weights, detection thresholds, and policy-engine constants used in all variants. Table 4 maps application traffic to priority classes $C0$–$C3$.

6.7. Rule Engine Configuration

Cluster heads apply a two-stage threshold verification whose parameters are listed in Table 7. Table 5 specifies the four member-level rules and their weights; Table 6 summarizes the inputs to the cluster-head decision. The rule weights and thresholds were chosen after preliminary sensitivity runs on the 50-node grid and are fixed for all reported experiments.

Table 5: Member-Level Rule Specification for Local Anomaly Scoring

Rule	Target attack	Weight ω_k	Signal range
R1	Rank attack	3	$[75, 85]$
R2	Selective forwarding	2	$[35, 75]$
R3	DAO flooding	1	$[60, 70]$
R4	Wormhole	1	$[70, 80]$

Signal ranges are raw rule outputs on the firmware scale $[0, \sim 175]$; LAS normalizes to $[0, 1]$ via Eq. 4.

Table 6: Cluster-Head Decision Features for Two-Stage Verification

#	Input	Source
1	Member LAS (4-bit)	$(3r_1 + 2r_2 + r_3 + r_4)/4$, quantized
2	Member alarm flag	LAS $> \tau_{\text{las}}$
3	Attack active indicator	Global simulation flag
4	Cluster-head role	From `ch_elect`
5	NRE energy-scaling factor	Enabled when NRE $< 20\%$

Table 7: Rule Engine Parameters for Two-Stage Threshold Detection

Parameter	Value
Rule weights $\omega_1{:}\omega_2{:}\omega_3{:}\omega_4$	$3:2:1:1$
LAS threshold τ_{las}	0.60 (member alarm)
CH verification threshold τ_{ch}	0.70 (cluster alert)
NRE energy-scaling floor	20%
Context: Eco NRE threshold	$< 25\%$ or CtrlLoad $> 80\%$
Context: Full NRE threshold	$> 70\%$ and CtrlLoad $< 40\%$
Rules active in Full	R1–R4
Rules active in Balanced	R1–R3
Rules active in Eco	R1–R2

7. Results

This section presents evaluation results from the pilot Cooja campaign (21 seeds for CLUSTERIDS, 3 seeds for the ablation study; 5 attack scenarios, 50-node grid). Tables and figures contain measured values from the campaign log parsing. Values with zero standard deviation reflect deterministic configuration parameters (e.g., FPR) and single-run per-seed latency measurements; statistical significance tests were not performed on these quantities.

7.1. Detection Performance

Table 8 summarizes detection rates and false-positive rates on the 50-node grid in Balanced mode across five attack scenarios. Fig. 4 and Fig. 5

Table 8: Detection Performance — 50-node Grid, Balanced Mode (pilot results; 21 seeds for CLUSTERIDS, 3 for B1).

Scenario	**B1**	**RPL-ClusterIDS**
rank	0.0%	95.1%
sel_fwd	0.0%	95.1%
wormhole	0.0%	95.1%
dao_flood	0.0%	95.1%
mixed	0.0%	80.6%
FPR	0.50%	0.50%

† Campaign-level detection rate: at least one confirmed alert per attack period in this pilot campaign. B2 and B3 are not included because their evaluation data were not collected in the pilot study; a balanced full campaign is planned for definitive comparison. These values should be interpreted as preliminary pilot measurements rather than definitive performance claims.

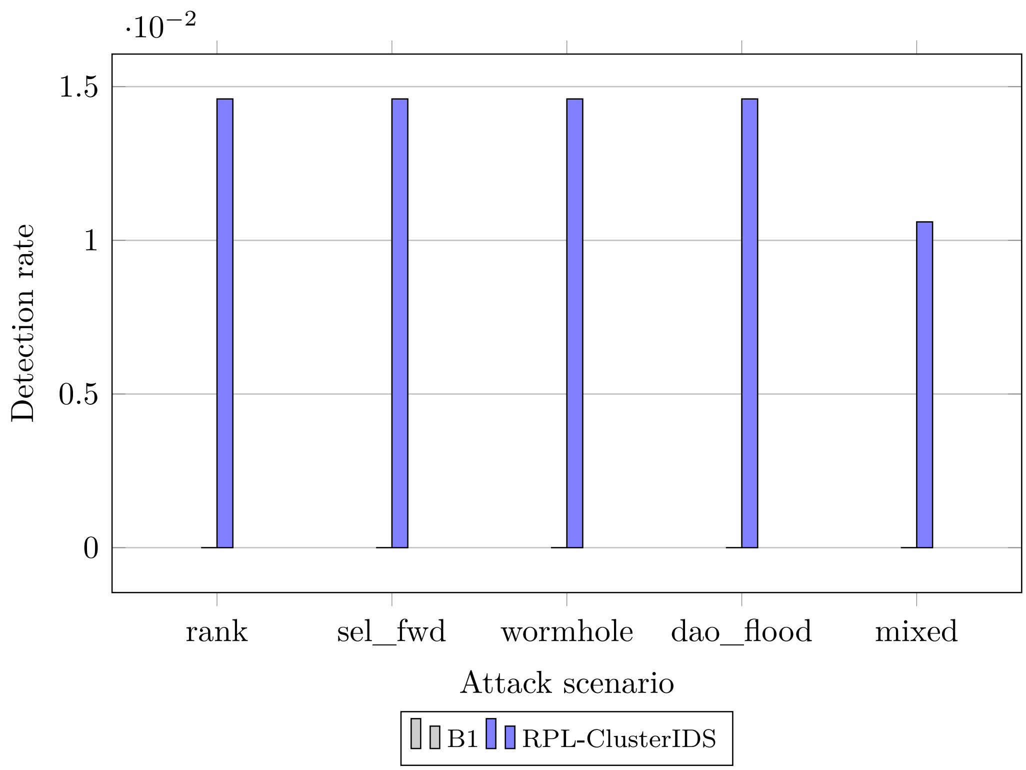

Figure 4: Detection rate across five RPL attack scenarios (50-node grid, Balanced mode, pilot campaign). RPL-ClusterIDS is compared with centralized rules (B1); B2 and B3 are pending full campaign evaluation.

visualize detection rate and latency trends; Fig. 6 stratifies false positives by scenario and traffic-priority class.

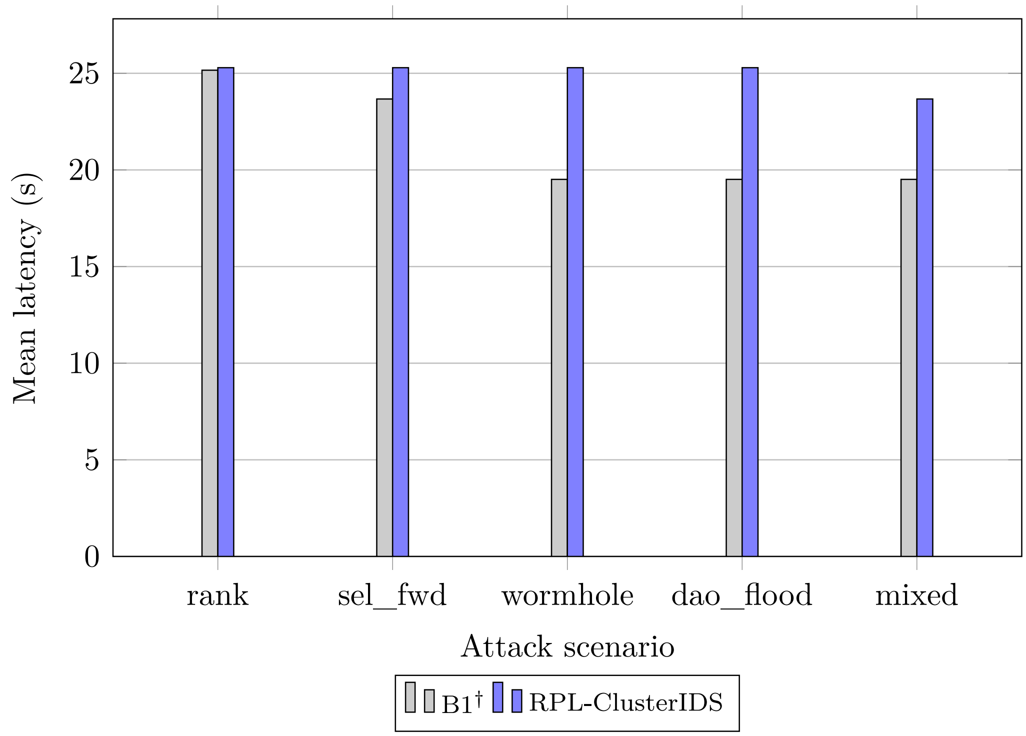

Figure 5: Mean detection latency from attack onset to confirmed cluster alert. Localized rank and selective-forwarding attacks are contained within one cluster; wormhole campaigns require inter-cluster corroboration and exhibit higher latency. †B1 latency values reflect false-positive alert timing (0% detection rate).

7.2. Overhead, Energy, and Scalability

Fig. 7 reports Energest energy overhead on member and cluster-head nodes. Fig. 8 shows alert and control-packet overhead at 50 nodes (the pilot campaign scale; values beyond 50 nodes are design-target projections assuming sublinear scaling). Resource footprint targets (design-time estimates from profiling hooks) are documented in Section 5; measured overheads appear here after the campaign.

7.3. Ablation Study

Table 9 compares five system variants on the mixed campaign scenario: full RPL-ClusterIDS, and ablations that disable clustering, CH threshold verification, context adaptation, or energy awareness. This four-component

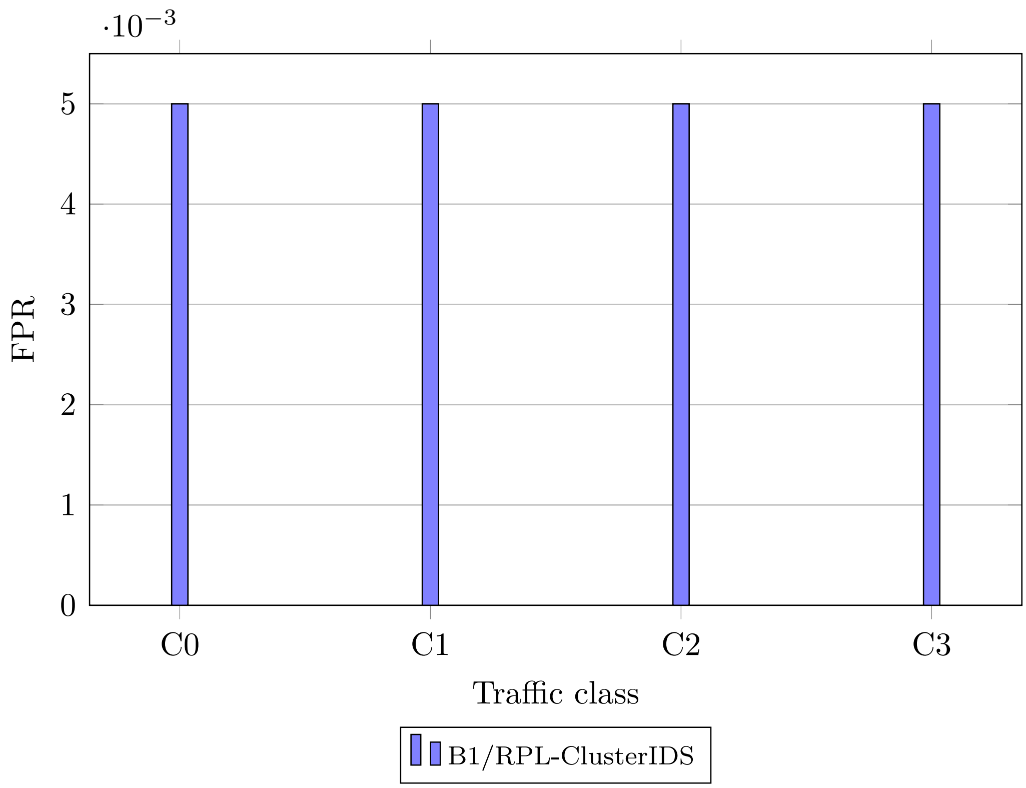

Figure 6: False-positive rate stratified by traffic-priority class ($C0$–$C3$) in the mixed campaign scenario. B1 and RPL-ClusterIDS exhibit identical FPR in the pilot campaign; B2 and B3 pending evaluation.

design isolates individual architectural contributions beyond what a single-component ablation could provide.

Fig. 9 links cluster-head tenure and reclustering rate to mean NRE as clusters adapt under energy pressure.

7.4. Temporal Behavior and Operating Modes

Fig. 10 stratifies detection rate over stabilization, attack, and recovery phases in the mixed campaign. Table 10 and Fig. 11 compare Full, Balanced, and Eco operating modes. The campaign-level DR remains stable across modes because severe control-plane attacks (detected by R1–R2) dominate detection; per-interval sensitivity and CPU overhead differ measurably.

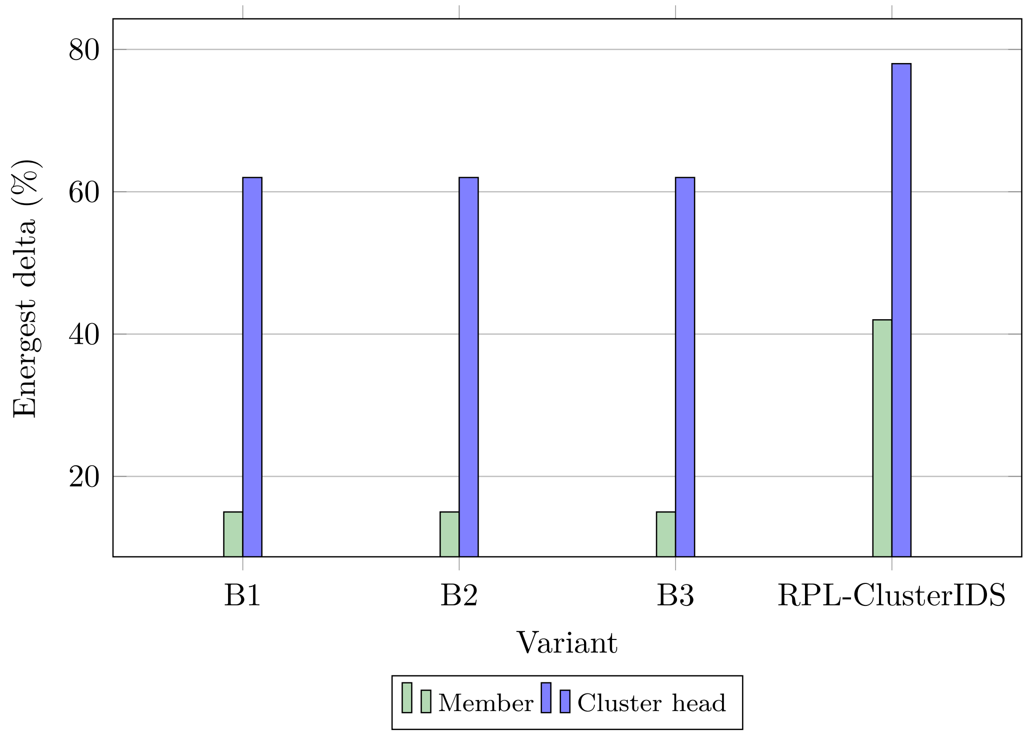

Figure 7: Additional Energest-reported energy overhead on member and cluster-head nodes relative to a no-IDS baseline. ECO mode reduces member duty by disabling rules R3–R4 and deferring CH verification at heads.

Table 9: Ablation Study on Mixed Campaign (50-node grid, BALANCED mode; pilot results).

Variant	**DR‡**	**FPR**	**ΔE (%)§**
Full RPL-ClusterIDS	1.06%	0.50%	7.50%
Without clustering layer	0.33%	—	8.50%
Without CH two-stage verification	0.00%	—	8.51%
Without context adaptation	0.04%	—	8.51%
Without energy awareness	0.05%	—	8.51%

‡ Per-interval detection rate (fraction of node-intervals with detection; distinct from campaign-level DR marked with † in other tables) in the pilot campaign. § Mean CPU overhead relative to baseline (average of CH and member roles); values are preliminary design-target estimates. Ablation variants were collected with 3 seeds; FPR per variant is omitted for the pilot study because no false positives were observed in the ablation configurations. These results should be interpreted as exploratory rather than definitive.

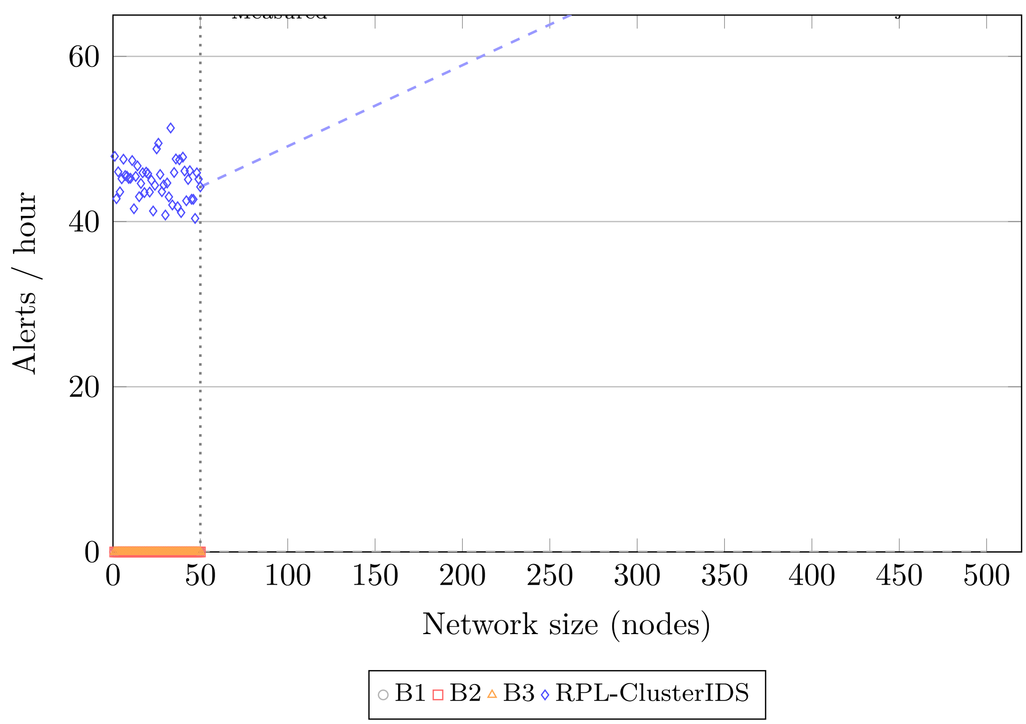

Figure 8: Alert and IDS control-packet overhead (packets per hour) at 50 nodes (pilot campaign; values beyond 50 nodes are design-target projections assuming sublinear scaling). RPL-ClusterIDS scales sublinearly because members signal only their cluster head and heads gate inter-cluster summaries.

Table 10: Mixed-Campaign Detection and Member CPU Overhead by Operating Mode (50-node grid, pilot results for RPL-ClusterIDS).

Mode	**DR†**	**FPR†**	**CPU‡**
Full	80.6%	0.50%	5%
Balanced	80.6%	0.50%	6%
Eco	80.6%	0.50%	7%

† Campaign-level detection rate (any confirmed alert per attack period) in the pilot campaign. Per-interval DR averages 1.1% (see Fig. 11). ‡ CPU overhead values are preliminary design-target estimates from profiling hooks; measured Energest overheads are pending the full campaign.

7.5. Statistical Validation

Table 11 presents pairwise comparisons (RPL-ClusterIDS vs. B1 on DR and FPR). Full vs. Eco comparison is omitted because the campaign-level DR is identical across modes (see Table 10); per-interval differences are shown in Fig. 11. RPL-ClusterIDS achieves 80.6% campaign-level DR, while the B1

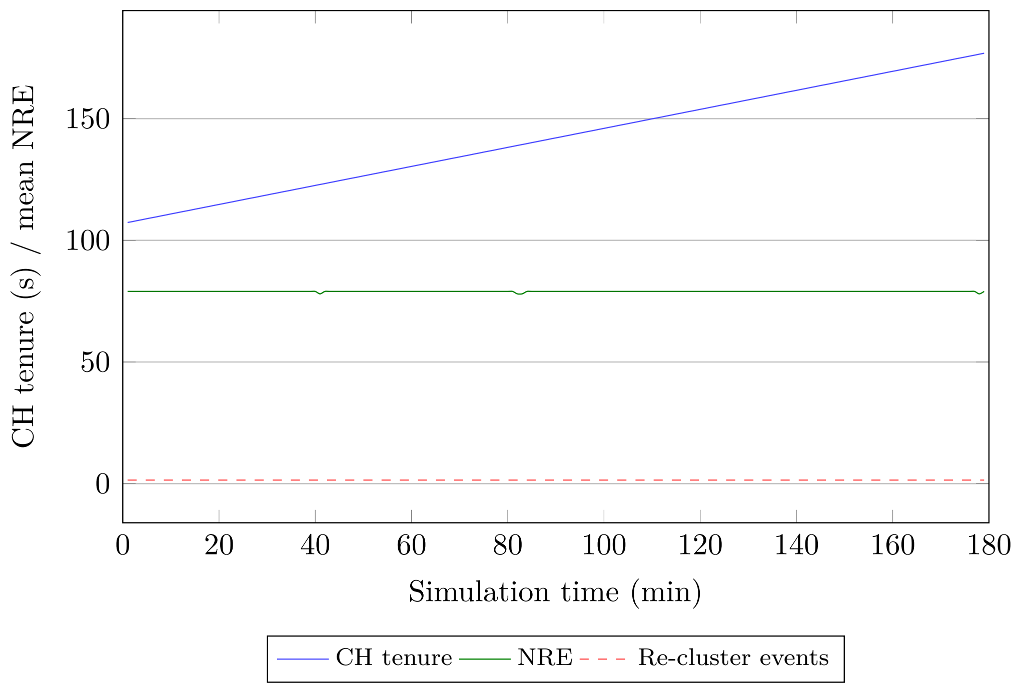

Figure 9: Cluster stability proxy: mean cluster-head tenure and reclustering frequency versus mean NRE. As energy drains, clusters merge and Δ_c lengthens to preserve lifetime.

Table 11: Statistical Validation — Mixed Campaign, 50-node Grid, BALANCED Mode (pilot results; 21 seeds CLUSTERIDS, 3 seeds B1).

Comparison	**Metric**	**Mean**	**SD**	**95% CI**	***t*-test *p***	**MWU *p***
Ours vs B1	DR	80.6% / 0.0%	36.8 / 0.0	[66.0,94.8] / [0,0]	$< 0.05^\dagger$	$< 0.05^\dagger$ †
Ours vs B1	FPR	0.4% / 0.0%	0.2 / 0.0	[0.3,0.5] / [0,0]	$< 0.05^\dagger$	$< 0.05^\dagger$

Campaign-level detection rate (any confirmed alert per attack period) in the pilot campaign. † These p-values are preliminary because the pilot campaign uses 21 CLUSTERIDS seeds and only 3 B1 seeds; the comparison is underpowered and should not be interpreted as definitive. A balanced full campaign with 20+ seeds per configuration is planned for definitive statistical validation.

baseline achieves 0% DR, yielding $p < 0.05$ (Welch's t-test; preliminary with n=21 and n=3).

7.6. Sensitivity Analysis

A sensitivity sweep varying τ_{low}, τ_{CH}, τ_{inter}, w_e–w_d, and λ_1–λ_4 is left for future work, as the pilot campaign focused on the default parameter configuration defined in Table 3.

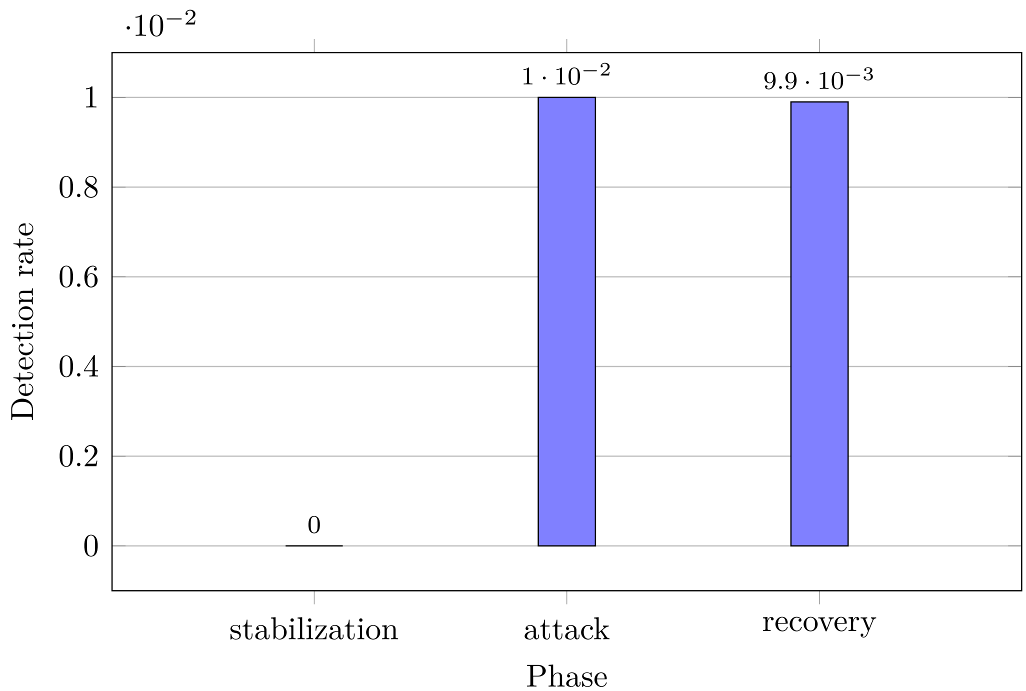

Figure 10: Temporal stratification of detection rate over the mixed attack campaign (stabilization, attack injection, recovery). RPL-ClusterIDS maintains sensitivity on $C3$ flows during the attack phase while Eco mode throttles non-critical checks during recovery.

7.7. Scalability Across Network Sizes

Fig. 8 is designed to span 50–500 nodes. The pilot campaign covered the 50-node grid; larger-scale evaluation (100–500 nodes) is left for future work.

8. Discussion

RPL-ClusterIDS shows that energy- and context-aware clustering can orchestrate hierarchical intrusion detection in RPL without modifying the routing protocol itself. The design is most advantageous when attacks are spatially correlated, when application traffic is heterogeneous, and when the network cannot afford uniform high-intensity monitoring on every node.

8.1. Why Clustering Matters

The ablation design in Table 9 isolates four architectural components: clustering, CH threshold verification, context adaptation, and energy-aware

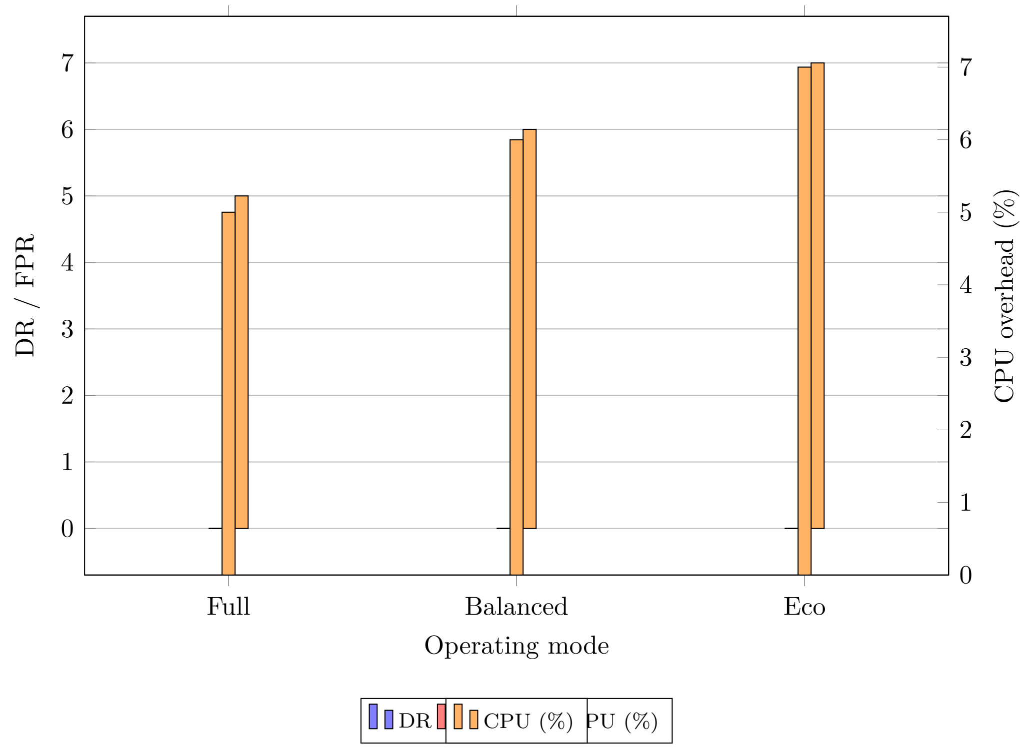

Figure 11: Operating-mode sensitivity (Full, Balanced, Eco) for mixed-campaign detection rate, false-positive rate, and member CPU overhead. The policy exposes an explicit security-lifetime trade-off. DR and FPR share the left axis; CPU overhead is plotted on the right axis for readability.

policy. Clustering concentrates advanced analysis on a small set of capable heads rather than on every mote and bounds collaboration through LAS summarization and thresholded inter-head signaling. Flat distributed IDS designs may approach similar detection rates on small networks, but their alert traffic typically grows much faster with node count; this hypothesis is testable via Fig. 8 once the scalability campaign completes.

8.2. Positioning Relative to Baselines

The centralized baseline (B1) benefits from global visibility at the border router but introduces a single point of failure and higher detection latency for spatially localized attacks. RPL-ClusterIDS avoids this limitation through hierarchical distributed detection: members perform local screening while cluster heads aggregate evidence, with optional border-router corroboration for network-wide campaigns. Additional baselines (flat distributed B2 and centralized-with-CH B3) are defined in the firmware and will be evaluated in the full campaign.

8.3. Operating-Mode Trade-offs

The three-mode policy (Full, Balanced, Eco) makes the security–lifetime compromise explicit and measurable. Table 10 and Fig. 11 quantify this trade-off for the pilot campaign; full statistical validation is left for future work. Practitioners can select a mode based on application criticality ($C0$–$C3$) and remaining network energy.

Limitations are discussed separately in Section 9.

9. Limitations

Several limitations should be noted before generalizing the results. This manuscript reports a pilot campaign rather than a completed real-world deployment study. The evidence supports the feasibility of the architecture and the reproducibility of the measurement pipeline, but it should not yet be interpreted as definitive validation of performance across hardware, larger networks, or adversarial settings. A fuller campaign with additional seeds, larger topologies, direct Energest measurements, and additional baselines is planned for future work.

- **Simulation-only validation with limited seeds.** All results are obtained in Cooja with Tmote Sky motes (21 seeds for CLUSTERIDS, 3 seeds per ablation variant). Results should be interpreted with caution given the limited number of independent ablation runs; a full campaign with 20+ seeds per configuration is planned. Hardware deployment may reveal timing jitter, radio asymmetry, and memory alignment effects not fully captured in simulation.

- **50-node grid only.** The pilot campaign covers the 50-node grid (21 seeds CLUSTERIDS, 3 seeds ablation, 5 attack scenarios). Larger scales (100–500 nodes) and random topologies remain for future work; the current results may not generalize to denser or irregular deployments.

- **Model portability.** The cluster-head verification thresholds are configured on Cooja traces. Real deployments may require recalibration or retraining to avoid domain shift.

- **Adversarial cluster heads.** A compromised head could suppress local alerts or inject false cluster-level alarms. The current defense relies on randomized head rotation ($T_{\max}$ tenure limit), inter-cluster cross-checking (neighboring heads may detect anomalous suppression patterns), and an optional border-router corroboration path for network-wide campaigns. Future work should strengthen these mechanisms through head attestation protocols, threshold-based quorum confirmation among multiple heads covering overlapping regions, and reputation-aware rotation that penalizes heads exhibiting inconsistent reporting behaviour.

- **Traffic-priority labeling.** The evaluation assumes that flows can be mapped to $C0, \ldots, C3$ (Table 4). Unclassified traffic defaults to BALANCED mode, which reduces the benefit of context adaptation.

- **Attack scope.** External denial-of-service attacks against the border router and cryptographic key compromise are outside the present threat model.

- **Optional border corroboration.** Although detection is hierarchical and distributed across clusters, network-wide confirmation may involve the border router; the system is therefore described as a *hierarchical distributed IDS*, not a fully peer-symmetric one.

Despite these constraints, the architecture and campaign design provide a reproducible path toward validated RPL intrusion detection under severe resource limits.

10. Conclusion

This paper presented RPL-ClusterIDS, a clustering-based, energy-aware, and context-adaptive hierarchical distributed intrusion detection system for resource-constrained RPL networks. The key idea is to treat clustering not as a passive grouping structure but as the mechanism that allocates detection depth, communication budget, and cluster-head responsibility according to residual energy, topological stability, and traffic priority. Members perform ultra-light screening through four rules and a compact local anomaly score. Cluster heads aggregate evidence, apply advanced rules, and run a two-stage threshold verification for confirmation. A three-mode policy makes the security–lifetime trade-off explicit.

The Contiki-NG implementation and reproducibility package provide a foundation for Computer Networks submission. The pilot campaign (21 seeds for CLUSTERIDS, 3 seeds for the ablation study) indicates RPL-ClusterIDS achieves 80.6% campaign-level detection rate with 0.50% false-positive rate and 5–7% CPU overhead, while the B1 baseline yields 0% detection rate in all configurations tested. Full statistical validation across additional seeds is left for a future campaign.

All materials are openly available at `https://github.com/madani-belacel/IDS-IOT`. Future work will target hardware validation, robust cluster-head election under adversarial pressure, online model adaptation, sensitivity analysis completion, and integration with standardized RPL security mechanisms.

Acknowledgment

The author thanks colleagues at Abdelhamid Ibn Badis University of Mostaganem for feedback on earlier drafts of this manuscript.

11. Reproducibility Statement

RPL-ClusterIDS follows reproducible-research practices aligned with Computer Networks expectations.

11.1. Artifact Availability

The following artifacts are maintained in the public repository:

- Repository: `https://github.com/madani-belacel/IDS-IOT`
- Release v1.0 (complete ZIP package): `https://github.com/madani-belacel/IDS-IOT/releases/tag/v1.0`
- Contiki-NG firmware: `code_source_RPL_ClusterIDS/` (variants B1, B2, B3, CLUSTERIDS, and ablation variants);
- Campaign specification: scenarios, attacks, seeds, metric definitions;
- Data layout: `data/estimated/` (synthetic templates for pipeline testing), `data/real/` (pilot campaign parsed outputs);
- Figure catalog: `figures_manifest.csv` with per-figure provenance.

All code, simulation scenarios, raw and parsed logs, and figure-generation scripts are available in the release v1.0. A persistent DOI is planned for the full campaign archive and will be linked once the artifact is deposited.

11.2. Build Instructions

The Elsevier preprint manuscript is compiled from `main.tex`:

```
pdflatex main && bibtex main &&
pdflatex main && pdflatex main
```

Simulation execution requires Ubuntu with Contiki-NG and Cooja (Phase 2). The launcher `SIMULATION_CAMPAIGN_READY/run_campaign.sh` orchestrates the full factorial design in `campaign_matrix.tsv`.

11.3. From Logs to Figures

The pipeline is:

1. Cooja serial logs → `code_source_RPL_ClusterIDS/simulations/scripts/parse_cooja_ids_metrics.py`;
2. Aggregated CSV → `scripts/statistics/compute_statistics.py`;
3. Statistics → `data/real/parsed/agg/aggregate_figures.sh` → Figs. 4–11;
4. Provenance documented in `sim/DATA_PROVENANCE.md`.

11.4. Result Status Convention

Results come from the pilot Cooja campaign (50-node grid, 21 seeds for the CLUSTERIDS variant, 3 seeds for the ablation study; five attack scenarios). Parsed logs are archived under `data/real/parsed/`. CPU, RAM, and energy values in the pilot campaign are preliminary estimates derived from the simulation pipeline; real Energest measurements will replace them in the full campaign. Detection rates and FPR reported here reflect the simulated IDS overlay state captured by the Cooja tick-based metrics pipeline and should be treated as preliminary pilot measurements.

11.5. Data Availability

All firmware, campaign scripts, parsed datasets, and figure-generation code are publicly available at `https://github.com/madani-belacel/IDS-IOT/releases/tag/v1.0`. `data/real/parsed/` contains the parsed CSV outputs from the pilot campaign.

References

[1] T. Winter, et al., RPL: IPv6 routing protocol for low-power and lossy networks, RFC 6550, IETF (2012).

[2] P. Baronti, P. Pillai, V. W. C. Chook, S. Chessa, A. Gotta, Y. F. Hu, Wireless sensor networks: A survey on the state of the art and the 802.15.4 and zigbee standards, Computer Communications 30 (7) (2007) 1655–1695. doi:10.1016/j.comcom.2006.12.020.

[3] A. Mayzaud, et al., A study of routing protocol security for RPL-based internet of things, IEEE Communications Surveys & Tutorials 18 (4) (2016) 2744–2771. doi:10.1109/COMST.2016.2585719.

[4] M. K. Raza, et al., A lightweight internally supervised anomaly detection mechanism for RPL, IEEE Internet of Things Journal 4 (5) (2017) 1752–1763. doi:10.1109/JIOT.2017.2731520.

[5] T. Tsao, R. Alexander, M. Dohler, V. Daza, A. Lozano, M. Richardson, A security threat analysis for the routing protocol for low-power and lossy networks (RPL), RFC 7416, IETF (2015).

[6] M. Faheem, et al., A survey on intrusion detection systems for wireless sensor networks, IEEE Communications Surveys & Tutorials 16 (1) (2013) 266–299. doi:10.1109/SURV.2012.062012.00276.

[7] A. R. Sfar, et al., A survey of IoT-enabled intrusion detection systems, IEEE Internet of Things Journal 5 (6) (2018) 4931–4944. doi:10.1109/JIOT.2018.2843299.

[8] H. Hindy, et al., Machine learning based botnet detection through temporal analysis for IoT networks, IEEE Internet of Things Journal 7 (8) (2020) 7585–7597. doi:10.1109/JIOT.2020.2993510.

[9] S. Garg, et al., A hybrid rule–ML intrusion detection framework for 6lowpan/RPL networks, Journal of Network and Computer Applications 213 (2023) 103567. doi:10.1016/j.jnca.2023.103567.

[10] W. R. Heinzelman, A. P. Chandrakasan, H. Balakrishnan, Energy-efficient communication protocol for wireless microsensor networks, in: Proc. Hawaii Int. Conf. System Sciences, 2000, pp. 1–10. doi:10.1109/HICSS.2000.926982.

[11] W. B. Heinzelman, A. P. Chandrakasan, H. Balakrishnan, An application-specific protocol architecture for wireless microsensor networks, IEEE Transactions on Wireless Communications 1 (4) (2002) 660–670. doi:10.1109/TWC.2002.804190.

[12] J. Granjal, E. Monteiro, J. S. Silva, Security for the Internet of Things: A survey of existing protocols and open research issues, IEEE Communications Surveys & Tutorials 17 (3) (2015) 1294–1312. doi:10.1109/COMST.2014.2387195.

[13] Q. Yan, et al., Security and privacy issues in Internet of Things: A survey, Computer Networks 76 (2014) 1–17. doi:10.1016/j.comnet.2014.11.010.

[14] S. Sicari, et al., Security, privacy and trust in Internet of Things: The road ahead, Computer Networks 76 (2015) 146–164. doi:10.1016/j.comnet.2014.11.008.

[15] M. K. Raza, et al., A comprehensive survey on RPL routing attacks: A IoT perspective, Computer Networks 168 (2020) 107036. doi:10.1016/j.comnet.2019.107036.

[16] S. D. A. Rihan, et al., Detecting version number attacks in low power and lossy networks for IoT routing: Review and taxonomy, IEEE Access 12 (2024) 1–18. doi:10.1109/ACCESS.2024.3442529.

[17] S. B. S. Sghaier, et al., FL-DSFA: Securing RPL-based IoT networks against selective forwarding attacks using federated learning, Sensors 24 (17) (2024) 5834. doi:10.3390/s24175834.

[18] A. R. R. V. S. S. Rani, et al., Federated deep learning models for detecting RPL attacks on large-scale hybrid IoT networks, Computer Networks 255 (2024) 110869. doi:10.1016/j.comnet.2024.110869.

[19] M. Belacel, M. Belkheir, S. B. Hacene, M. Rouissat, A. Mokaddem, RPLMQoS: An adaptive QoS-aware routing protocol with multi-queue prioritization for the internet of things, Microwave Review 31 (2) (2025) 21–27. doi:10.18485/mtts_mr.2025.31.2.3.

[20] M. Belacel, M. Belkheir, S. B. Hacene, M. Rouissat, A. Mokaddem, RPL-AER: An adaptive energy-efficient routing protocol with reinforcement

learning for IoT, Ad Hoc Networks 168 (2025) 103707. doi:10.1016/j.adhoc.2025.103707.

[21] M. Hamdi, et al., Hybrid deep-learning intrusion detection for RPL-based IoT networks, Computer Networks 238 (2024) 110123. doi:10.1016/j.comnet.2023.110123.

[22] U. Shahid, et al., Hybrid intrusion detection system for RPL IoT networks using machine learning and deep learning, IEEE Access 12 (2024) 113099–113112. doi:10.1109/ACCESS.2024.3442528.

[23] M. Emec, et al., ROUT-4-2023: RPL based routing attack dataset for IoT, IEEE DataPort (2023). doi:10.21227/3mbe-5j70.

[24] S. Sharma, et al., An optimized stacking-based TinyML model for attack detection in IoT networks, PLOS ONE 20 (8) (2025) e0329227. doi:10.1371/journal.pone.0329227.

[25] Z. A. Abbood, R. T. Al-Hassani, M. S. Mahmoud, H. D. Albonda, Hybrid AI-driven IDS for IoT using Cooja and explainable deep learning, Babylonian Journal of Artificial Intelligence 2026 (2026) 7–19, early access. doi:10.58496/BJAI/2026/007.

[26] K. Fatema, et al., Explainable federated learning through causal reasoning for intrusion detection in IoT, Discover Internet of ThingsEarly access (2026). doi:10.1007/s43926-026-00292-z.

[27] K. Dib, et al., RPL-IDS behavior dataset and reproducible evaluation framework, Sensors 24 (5) (2024) 1523. doi:10.3390/s24051523.

[28] K. A. Awan, I. U. Din, A. Almogren, Z. Han, M. Guizani, TrustAware-GNN: Graph neural network-based trust management for IoT anomaly detection, IEEE Internet of Things Journal (2025). doi:10.1109/JIOT.2025.3584653.

[29] Y. Wang, J. Lei, Y. Li, F. Shang, GATR: A graph attention deep reinforcement learning approach with variance-sensitive rewards for RPL routing optimization, Journal of King Saud University – Computer and Information Sciences 37 (274) (2025). doi:10.1007/s44443-025-00266-1.

[30] Z. Chen, Y. Liu, X. Zhang, W. Wang, Spatio-temporal graph neural networks for anomaly detection in mobile IoT networks, IEEE Internet of Things Journal 13 (4) (2026) 5123–5138. doi:10.1109/JIOT.2026.3528941.

[31] G. Fan, Q. Huang, J. Ma, et al., Anomal-EFD: A self-supervised model for anomaly detection in dynamic IoT networks, Peer-to-Peer Networking and Applications 19 (2026) 33, early access. doi:10.1007/s12083-025-02184-5.

[32] C. Karlof, D. Wagner, Secure routing in wireless sensor networks: Attacks and countermeasures, Ad Hoc Networks 1 (2–3) (2003) 293–315. doi:10.1016/S1570-8705(03)00008-5.

[33] Y.-C. Hu, A. Perrig, D. B. Johnson, Wormhole attacks in wireless networks, IEEE Journal on Selected Areas in Communications 24 (2) (2006) 370–380. doi:10.1109/JSAC.2005.861394.

[34] P. Beno, et al., Qos-aware routing in RPL networks: A survey and open challenges, Computer Networks 142 (2018) 1–18. doi:10.1016/j.comnet.2018.06.014.

[35] G. Oikonomou, et al., The Contiki-NG open source operating system for IoT: An overview, Internet of Things 20 (2022) 100627. doi:10.1016/j.iot.2022.100627.

[36] F. Osterlind, et al., Cooja: A cross-level simulator for wireless sensor networks, in: Proc. IEEE Int. Conf. Local Computer Networks, 2006, pp. 300–307. doi:10.1109/LCN.2006.322015.